LangGraph-E-M-Set1

20 explained answers | Python Graph API | Official documentation checked October 28, 2026

01. Purpose of LangGraph

Answer B: Coordinate a stateful workflow with branches, loops, and human review.

LangGraph provides the execution structure for stateful workflows. A credit-report process can retrieve evidence, draft sections, evaluate them, and route weak sections back for revision. Nodes can contain ordinary Python logic or model calls; the graph controls how the work progresses.

Why the other choices are wrong

A. Embedding storage and similarity search belong to a retrieval backend, which a graph can call.

C. Payload validation is typically handled by a schema or validation library, such as Pydantic.

D. An HTTP framework such as FastAPI exposes endpoints; those endpoints can invoke a graph.

Remember: Use LangGraph to coordinate work and state across multiple steps.

Official references: [1]

02. Shared state

Answer D: It is structured data that nodes read and update during execution.

Graph state carries the information needed across steps. For example, a retrieval node can write evidence into state and a drafting node can read that evidence later. The schema describes the available fields and their update behavior. State does not need to consist only of chat messages.

Why the other choices are wrong

A. Execution uses a call stack, but that stack is not the graph state schema.

B. Nodes choose how to construct prompts; state is not automatically a prompt.

C. A vector index is an external retrieval resource, not the shared state itself.

Remember: State is the structured information that moves through the workflow.

Official references: [2]

LangGraph-E-M-Set1

20 explained answers | Python Graph API | Official documentation checked October 28, 2026

03. Node updates

Answer A: {"answer": "Approved"}

A normal StateGraph node can return a dictionary containing only the fields it updates. Returning the answer field leaves question and evidence unchanged. This makes each node responsible for its own contribution without requiring it to reconstruct every field in the shared state.

Why the other choices are wrong

B. A bare string does not identify which state field should be updated.

C. Explicitly supplying None also updates the other fields; it does not preserve their values.

D. Changing a local variable does not communicate an answer update to the graph runtime.

Remember: Return a partial state update containing the fields the node changes.

Official references: [3]

04. Compilation

Answer C: An executable graph that can be invoked or streamed.

StateGraph is a builder for describing the workflow. Compilation turns that description into an executable graph and performs structural checks. The resulting graph supports execution methods such as invoke() and stream(). Running the workflow is a separate action that supplies the input.

Why the other choices are wrong

A. Compilation does not build a retrieval index.

B. Compiling prepares the graph; it does not execute its nodes with invented input.

D. Serving the graph over HTTP requires separate application or deployment infrastructure.

Remember: Build the graph, compile it, then invoke or stream it.

Official references: [3]

LangGraph-E-M-Set1

20 explained answers | Python Graph API | Official documentation checked October 28, 2026

05. Entry and exit

Answer B: The workflow entry point and a terminal destination.

An edge from START identifies where graph execution begins. Routing to END marks completion of that path. For a simple linear workflow, START can connect to retrieval, retrieval to drafting, and drafting to END. These markers describe graph control flow rather than application data.

Why the other choices are wrong

A. Message ordering belongs to a messages field and its updates.

C. Database lifecycle management is separate from these graph markers.

D. Model token generation is internal to a model call and is not defined by START or END.

Remember: START and END express graph-level entry and termination.

Official references: [2]

06. Conditional routing

Answer D: A conditional edge whose routing function selects the destination.

A conditional edge evaluates routing logic after its source node runs. The routing function can inspect the updated quality result and choose one destination for this scenario. A normal outgoing edge always schedules its destination when the source completes; it is not an either-or decision.

Why the other choices are wrong

A. Two unconditional outgoing edges schedule both destinations rather than selecting one.

B. A reducer combines field values; it does not decide which node runs next.

C. A thread ID identifies persisted execution state, not a branch in the workflow.

Remember: Use conditional routing when the next step depends on state.

Official references: [4]

LangGraph-E-M-Set1

20 explained answers | Python Graph API | Official documentation checked October 28, 2026

07. Message history

Answer A: It updates the matching message; messages with new IDs are appended.

add_messages merges message updates using message IDs. A new ID adds a message, while an existing ID allows that message to be replaced. This is useful when correcting a prior message without adding a second copy of it. It is more specialized than simple list concatenation.

Why the other choices are wrong

B. Unconditional concatenation would duplicate matching IDs; add_messages handles those IDs specially.

C. Existing messages can be updated, so the history is not immutable in this sense.

D. The reducer does not define alphabetical sorting of message IDs.

Remember: add_messages is an ID-aware merge, not just list append.

Official references: [5]

08. Checkpointing

Answer C: Persist graph execution state so a thread can continue or resume.

A checkpointer records a thread's graph state and execution information at checkpoints. This supports conversation continuity, inspection, and resuming interrupted work. It preserves execution context; it does not judge the truth of an answer or replace application-level evaluation.

Why the other choices are wrong

A. Factual evaluation requires explicit evaluation logic and evidence.

B. Checkpoints contain execution state, not merely the source-code files.

D. Checkpointing does not rewrite the implementation of node functions.

Remember: A checkpointer preserves where a thread is and the state needed to continue.

Official references: [6]

LangGraph-E-M-Set1

20 explained answers | Python Graph API | Official documentation checked October 28, 2026

09. Thread identity

Answer A: {"configurable": {"thread_id": "report-42"}}

The checkpointer uses thread_id under the configurable section of the execution config. Calls using that thread identity can access its checkpointed state. Metadata and tags can help describe a run, but descriptive labels do not substitute for the checkpointer's thread key.

Why the other choices are wrong

B. A metadata field with that name does not set the required configurable thread identity.

C. Naming a run is different from identifying a persisted thread.

D. A tag labels a run and does not select a checkpointed thread.

Remember: Pass thread_id in config["configurable"] when using a checkpointer.

Official references: [7]

10. Streaming state updates

Answer D: Node-produced state updates as graph execution progresses.

The updates stream mode exposes the changes produced by nodes as execution progresses. It is useful for seeing when retrieval adds evidence or drafting produces an answer. Other stream modes serve different purposes: values provides full state values, while messages exposes model message chunks with metadata.

Why the other choices are wrong

A. Model token or message-chunk streaming is associated with messages mode.

B. Updates mode provides incremental updates, not just a single final-state result.

C. Streaming events can feed a user interface, but this mode does not create tracing dashboards.

Remember: updates shows node updates; values shows full state; messages shows model output chunks.

Official references: [9]

LangGraph-E-M-Set1

20 explained answers | Python Graph API | Official documentation checked October 28, 2026

11. Reducers and sequential updates

Answer B: {"count": 7, "notes": ["retrieved", "drafted"]}

The count field has no custom reducer, so its new value replaces 4 with 7. The notes field uses operator.add, which concatenates lists: ["retrieved"] + ["drafted"] becomes ["retrieved", "drafted"]. Each state field follows its own update rule; a reducer on one field does not apply to all fields.

Why the other choices are wrong

A. The integer is not added automatically, and the list has a concatenation reducer.

C. The notes value is right, but count should be replaced with 7 rather than summed.

D. The count value is right, but notes should retain the old entry and append the new one.

Remember: Reducers are defined per field; without one, a single update replaces the prior value.

Official references: [4]

12. Parallel writes

Answer C: A concurrent-update error because the field cannot merge both writes.

An ordinary state field does not define how to combine multiple writes in one superstep. LangGraph reports a conflicting update rather than silently selecting a winner. If the intended behavior is to collect both lists, annotate the field with an appropriate reducer, such as operator.add for list concatenation.

Why the other choices are wrong

A. List values do not imply an automatic concatenation reducer.

B. There is no automatic last-writer-wins policy for this conflicting update.

D. The scheduler does not solve the conflict by sorting and serializing nodes by name.

Remember: Parallel writers to a shared field need a compatible aggregation rule.

Official references: [10]

LangGraph-E-M-Set1

20 explained answers | Python Graph API | Official documentation checked October 28, 2026

13. Dynamic worker fan-out

Answer A: Return Send("write_section", section_input) objects from routing logic.

Send identifies a destination node and the input for a particular task. Routing can create one Send per discovered section, so the number of worker tasks is determined at runtime. The workers can contribute to a shared result field with an appropriate reducer, after which a later step assembles the report.

Why the other choices are wrong

B. A static edge schedules its destination; it does not automatically map over a list in state.

C. A reducer combines state updates and does not schedule worker tasks.

D. Thread IDs identify separate persisted executions; they do not automatically create a parent-worker join.

Remember: Use Send for dynamic fan-out with distinct input for each task.

Official references: [11]

14. Parallel branch join

Answer D: builder.add_edge(["section_a", "section_b"], "assemble")

Passing a list of source node names to add_edge expresses a join: the destination waits for all listed sources. This is useful when assembling independent report sections. An edge from only one source cannot establish that the other source has also finished.

Why the other choices are wrong

A. This edge depends only on section_a, so it does not require section_b to finish.

B. This edge depends only on section_b, so it does not require section_a to finish.

C. This controls what happens after assemble and does not establish its prerequisites.

Remember: A list of sources in add_edge expresses an all-sources join.

Official references: [3]

LangGraph-E-M-Set1

20 explained answers | Python Graph API | Official documentation checked October 28, 2026

15. Updating and routing together

Answer B: Command(update={"approved": True}, goto="publish")

Returning Command from a node can combine a state update with a routing decision. Here, update writes approved and goto schedules publish. The no-static-edges assumption matters: Command routing does not remove existing static outgoing edges, so an unintended static edge could schedule additional work.

Why the other choices are wrong

A. This updates state but does not itself select publish as the next node.

C. Send is used to dispatch task inputs through routing; this is not the normal node return for updating shared state and routing together.

D. resume supplies external input to a paused interrupt; it does not express this update-and-route operation.

Remember: Return Command(update=..., goto=...) to change state and choose the next node together.

Official references: [12]

16. Resuming human review

Answer C: graph.invoke(Command(resume=True), config=original_thread_config)

Command(resume=True) delivers the external decision to the waiting interrupt. The config must identify the original thread so the runtime finds its saved execution. On resumed execution, the stored resume value becomes the return value of interrupt(), allowing approved to receive True.

Why the other choices are wrong

A. A new thread identifies different state, and a plain state update is not the interrupt-resume protocol.

B. A new thread does not identify the paused execution or supply its approval value.

D. goto is for routing from nodes; it is not a substitute for providing an interrupt response.

Remember: Resume an interrupt with Command(resume=...) and the same thread_id.

Official references: [8]

LangGraph-E-M-Set1

20 explained answers | Python Graph API | Official documentation checked October 28, 2026

17. Interrupt replay and side effects

Answer A: The code before interrupt() can run again, creating a duplicate ticket.

Resuming an interrupted node restarts that node from its beginning. Code before the interrupt may therefore execute again. An external action should be idempotent, for example by using a stable request key that the ticket service deduplicates. A graph checkpoint alone cannot guarantee exactly-once effects in another system.

Why the other choices are wrong

B. LangGraph does not automatically roll back actions performed by an external service.

C. Resume does not disable functions or prevent later execution.

D. A checkpoint records workflow state; it does not make a remote ticket API transactional with the graph.

Remember: Design side effects for replay; use idempotency where duplicate execution would matter.

Official references: [8]

18. Durable checkpoint storage

Answer D: Use a persistent checkpointer and resume with the same thread identity.

InMemorySaver keeps checkpoints in process memory, which is lost when that process exits. A database-backed checkpointer can retain the required state across restarts. Reusing the thread identity is still necessary to find the saved execution, but an identifier cannot recover data that was never stored durably.

Why the other choices are wrong

A. The step limit changes execution bounds, not the lifetime of stored checkpoints.

B. Streaming exposes execution output; it does not make in-memory checkpoints survive shutdown.

C. A matching ID is insufficient if the checkpoint data disappeared with the process.

Remember: Resuming after a restart requires durable storage as well as the correct thread identity.

Official references: [15]

LangGraph-E-M-Set1

20 explained answers | Python Graph API | Official documentation checked October 28, 2026

19. Bounded revision loops

Answer B: Track revision attempts, exit or escalate at a limit, and retain a recursion-limit safeguard.

A revision counter and explicit routing rule create an application-level stopping policy, such as escalating after three unsuccessful revisions. The recursion limit adds a runtime safeguard against excessive graph steps. Hitting that limit raises an error; it does not by itself produce an approved report or a meaningful escalation.

Why the other choices are wrong

A. An unbounded failure branch can loop indefinitely until a runtime limit is reached.

C. Raising the limit does not correct a missing stopping condition.

D. Publishing every draft bypasses the quality requirement rather than managing failed revisions.

Remember: Use explicit termination logic; treat recursion_limit as an execution safeguard.

Official references: [13]

20. Memory across threads

Answer C: Store preferences in a Store under a user-scoped namespace and load them when needed.

A Store holds application data that can be accessed across threads. A user-scoped namespace can organize preferences independently of a particular report execution. Each report can keep its own checkpointed state while loading the same preferences. The application still needs to implement the reads, writes, and access checks.

Why the other choices are wrong

A. Thread checkpoints do not automatically copy themselves into unrelated new threads.

B. A single global thread mixes execution state and is unsuitable for isolating users and reports.

D. Local variables are transient and do not provide persistent cross-thread memory.

Remember: Use checkpointers for thread state and a Store for data shared across threads.

Official references: [14]

LangGraph-E-M-Set1

Source guide | Official LangChain / LangGraph Python documentation

References support the concepts behind the original practice questions. Code assumes standard imports and uses the Python StateGraph API. This set is independent practice material, not an official certification exam.

[1] LangGraph overview
https://docs.langchain.com/oss/python/langgraph/overview

[2] Graph API overview
https://docs.langchain.com/oss/python/langgraph/graph-api

[3] StateGraph reference
https://reference.langchain.com/python/langgraph/graph/state

[4] Use the graph API
https://docs.langchain.com/oss/python/langgraph/use-graph-api

[5] Message reducers reference
https://reference.langchain.com/python/langgraph/graph/message

[6] Persistence
https://docs.langchain.com/oss/python/langgraph/persistence

[7] Checkpointers
https://docs.langchain.com/oss/python/langgraph/checkpointers

[8] Interrupts
https://docs.langchain.com/oss/python/langgraph/interrupts

[9] Streaming
https://docs.langchain.com/oss/python/langgraph/streaming

[10] Concurrent state update error
https://docs.langchain.com/oss/python/langgraph/errors/INVALID_CONCURRENT_GRAPH_UPDATE

[11] Workflows and agents
https://docs.langchain.com/oss/python/langgraph/workflows-agents

[12] Types reference: Command and Send
https://reference.langchain.com/python/langgraph/types

[13] Graph recursion limit
https://docs.langchain.com/oss/python/langgraph/errors/GRAPH_RECURSION_LIMIT

[14] Stores
https://docs.langchain.com/oss/python/langgraph/stores

[15] Memory
https://docs.langchain.com/oss/python/langgraph/add-memory